

Lenguajes para Análisis de Datos II

Modulo 3

Objetivo

En esta unidad se exploran dos herramientas fundamentales para el procesamiento de datos en Python: la librería datetime y NumPy. La librería datetime ofrece funcionalidades para manejar fechas y horas de manera precisa, permitiendo operaciones aritméticas y comparaciones entre ellas. Por otro lado, NumPy es un pilar de la computación científica, proporcionando soporte para matrices multidimensionales y funciones matemáticas de alto rendimiento.

Ambas librerías, datetime y NumPy, son esenciales en el kit de herramientas de cualquier científico de datos o programador en Python, brindando capacidades fundamentales para el manejo y análisis de datos.



La librería `datetime` proporciona clases y funciones para trabajar con **fechas, horas y duraciones de tiempo**. Permite manejar información de fechas y horas de manera sencilla y precisa, lo que lo convierte en una herramienta útil para tareas relacionadas con el manejo del tiempo y los calendarios. Las librerías o bibliotecas son herramientas adicionales que expanden las capacidades del lenguaje de programación, en este caso Python. Son conjuntos de código ya escrito que ofrecen al programador una amplia variedad de funcionalidades para realizar tareas específicas, ahorrando tanto tiempo como esfuerzo. Las librerías permiten analizar, manipular y visualizar datos, desarrollar aplicaciones web, automatizar tareas o acceder a APIs.

Dentro de `datetime`, podemos encontrar algunas clases importantes:

- `datetime`: Representa una combinación de fecha y hora.
- `date`: Representa una fecha (año, mes y día) sin información de hora.
- `time`: Representa una hora del día (horas, minutos, segundos y microsegundos).
- `timedelta`: Representa una duración o diferencia entre dos puntos de tiempo.
- `tzinfo`: Representa información sobre la zona horaria.



Comandos y ejemplos

Creación de objetos datetime, date y time:

```
#Importar datetime, date y time del módulo datetime
```

```
from datetime import datetime, date, time
```

En este mismo apartado, podemos crear diferentes tipos de objetos:

```
# Objeto datetime
```

```
fecha_hora = datetime(2023, 8, 1, 12, 30, 0)
```

```
print(fecha_hora)
```

```
# Resultado: 2023-08-01 12:30:00
```

```
# Objeto date que sólo contenga la fecha
```

```
fecha = date(2023, 8, 1)
```

```
print(fecha)
```

```
# Resultado: 2023-08-01
```

```
# Objeto time que sólo tenga la hora
```

```
hora = time(12, 30, 0)
```

```
print(hora)
```

```
# Resultado: 12:30:00
```

Python Datetime



Obtención de fecha y hora:

```
# Importar fecha y hora desde el módulo datetime  
from datetime import datetime
```

```
# Obtener fecha y hora actual  
fecha_hora_actual = datetime.now()  
print(fecha_hora_actual)  
# Resultado: 2023-08-01 13:45:22.123456
```

**PYTHON
DATETIME
MODULE**



YoungWorks

Operaciones con timedelta: Podemos obtener una nueva fecha basándonos en la fecha actual.

```
# Importar la fecha y el timedelta del módulo datetime  
from datetime import datetime, timedelta
```

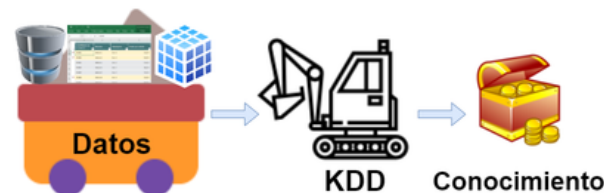
```
# Obtener la fecha y hora actual  
fecha_hora_actual = datetime.now()
```

```
# Crear un timedelta de 7 días  
duracion = timedelta(days=7)
```

```
# Sumar el timedelta de 7 días a la fecha actual para obtener la nueva
```

```
nueva_fecha = fecha_hora_actual + duracion  
print(nueva_fecha)
```

```
# Resultado: 2023-08-08 13:45:22.123456
```



d.strftime

En ocasiones, necesitamos la fecha y la hora en un formato específico y datetime nos permite organizarlas en el que requiramos. El método d.strftime nos devuelve la cadena que resulta de transformar la fecha con el formato indicado en la cadena formato.

```
# importar fecha y hora del módulo datetime
from datetime import datetime
```

```
# Obtener la fecha y hora actual
fecha_hora_actual = datetime.now()
```

```
# Formatear la fecha y hora en el formato que especifiquemos. En este
caso, hemos elegido año-mes-día hora-minuto-segundo
formato = "%Y-%m-%d %H:%M:%S"
fecha_hora_formateada = fecha_hora_actual.strftime(formato)
print(fecha_hora_formateada)
# Resultado: "2023-08-01 13:45:22"
```



- %Y (año completo)
- %y (últimos dos dígitos del año)
- %m (mes en número)
- %B (mes en palabra)
- %d (día)
- %A (día de la semana)
- %a (día de la semana abreviado)
- %H (hora en formato 24 horas)
- %I (hora en formato 12 horas)
- %M (minutos)
- %S (segundos)
- %p (AM o PM)
- %C (fecha y hora completas)
- %x (fecha completa)
- %X (hora completa)

Usos

La librería `datetime` es ampliamente utilizada en Python por la versatilidad que proporciona al programador. Algunos de los usos más comunes son:

- El manejo de datos de tiempo y fechas en aplicaciones de ciencias de datos, análisis de datos y visualización.
- La realización de cálculos y operaciones con fechas, como por ejemplo sumas, restas y comparaciones.
- El formateo y manipulación de cadenas de fechas y horas para una futura presentación en interfaces de usuario o almacenamiento en bases de datos.
- El trabajo con series de tiempo y programación relacionada con eventos temporales.

Además de los comandos y ejemplos que hemos visto, `datetime` ofrece una variedad de funcionalidades para trabajar con fechas, horas y duraciones.



¡Sigamos trabajando!